

KIDNEYVISION AI

Medical AI System for Renal Image Analysis

Technical & Project Documentation

Authors

Marouan Sellami
Mossab Oued Neimia
Souhibe Bakkali

Project Documentation

2026

Document Information & Technical Registry

Attribute	Technical Specification
Project Name	KidneyVision AI — Clinical Decision-Support Platform
Document Title	Comprehensive System Architecture, Deep Learning Pipeline & Production Report
Document Classification	Academic Engineering Thesis & Medical AI Technical Dossier
Version & Lifecycle	Release 2.4.0 (Production Stabilized) — March 2026
Project Authors	Marouan Sellami, Mossab Ouelid Neimia, Souhibe Bakkali
Automated Test Coverage	106 / 106 Tests Passing (E2E, Integration, Unit, Security, Fault-Tolerance)
Deep Learning Core	PyTorch 2.2+ (AMP Mixed Precision), Meta ConvNeXt-Tiny, MobileNetV3
Backend REST API	Laravel 12.0, PHP 8.2-FPM, Service Layer, DTO Pattern, SQLite Database
Frontend Clinical SPA	React 19.0, TypeScript, Vite, TailwindCSS, Dual-Layer Canvas Visualizer
AI Microservice	Flask 3.1, Gunicorn WSGI Worker Architecture, Thread-Safe GPU Daemon
DevOps & Containerization	Docker, Docker Compose, Nginx Edge Gateway, Automated Process Supervisors
Cloud Hosting Environment	Microsoft Azure (Region: Denmark East), VM Standard_B2als_v2 (Ubuntu 24.04 LTS)
Public Production Domain	http://kidneyvision.publicvm.com/
Source Code Repository	https://github.com/marouan-sell/kidneyVision

Table 1 — System metadata and technical specification registry.

Methodological Stance & Safety Definition

KidneyVision AI is engineered as an **investigative clinical decision-support and research platform**. Its objective is to accelerate tomographic (CT) and ultrasound pre-triage, alerting physicians to renal abnormalities (stones, cysts, tumors) in real time. The software is not an autonomous diagnostic device and does not replace the attending radiologist's review and formal diagnosis.

Table of Contents

PART I — INTRODUCTION AND PROJECT GENESIS	4
PART II — FIRST APPROACH AND WHY IT FAILED	6
PART III — COMPLETE AI REBUILD (SOTA ENGINE)	7
PART IV — EXPLAINABLE AI AND SURGICAL TARGETING	11
PART V — FULL-STACK PLATFORM IN MICROSERVICES	13
PART VI — CRITICAL BUGS AND PRODUCTION POST-MORTEM	14
PART VII — SECURITY, TESTING AND OPERATIONAL RELIABILITY	15
PART VIII — DOCKER CONTAINERIZATION AND AZURE CLOUD	16
PART IX — RESULTS SYNTHESIS, LIMITATIONS AND FUTURE WORK	17
PART X — FINAL CONCLUSION AND OFFICIAL LINKS	18

Note: Sub-chapters flow continuously within each main part to provide a cohesive, dense engineering document.

PART I — INTRODUCTION AND PROJECT GENESIS

Chapter 1 — Introduction to KidneyVision AI

1.1 System Definition and Functional Boundaries

KidneyVision AI is an enterprise-grade medical imaging decision-support and computer vision platform engineered specifically for the rapid pre-triage and analysis of axial renal computed tomography (CT) and B-mode ultrasonography scans. The system bridges the gap between deep learning research models and high-throughput emergency hospital workflows, packaging modern convolutional and vision transformer backbones into an audited, secure, and highly responsive clinical portal.

A foundational principle of this project is that KidneyVision AI operates strictly as an *investigative decision-support system* rather than an autonomous diagnostic authority. Medical imaging is characterized by high intra-class variability, patient-specific anatomical variations, and subtle radiological artifacts. The objective of the platform is therefore to act as a high-speed pre-triage filter—surfacing high-probability abnormalities, quantifying structural measurements, and providing visual heatmaps that enable attending physicians to prioritize critical cases during emergency shifts.

1.2 Inherent Difficulties in Renal Image Analysis

Applying automated computer vision to renal imaging presents challenges distinct from standard benchmark vision tasks:

- **High Noise and Variable Attenuation:** Renal parenchyma exhibits subtle attenuation gradients on unenhanced CT scans, making differentiation between fluid-filled lesions, solid neoplasia, and normal cortex challenging without intravenous contrast agents.
- **Anatomical Variability & Acquisition Artifacts:** Slice acquisition varies according to patient respiratory cycles, scanner calibration, and body habitus. Mobile digitized scans captured from printed films or monitors introduce perspective distortions and ambient illumination noise.
- **Catastrophic Cost of Silent Misclassification:** Unlike consumer vision applications where occasional misclassifications incur negligible penalties, a missed renal mass or false reassurance on an obstructing calculus can lead to irreversible nephron loss or acute uropsepsis.

1.3 Intended Users and Clinical Operational Profile

The software platform is engineered to serve four distinct clinical and research cohorts:

1. **Emergency Room Clinicians:** Managing acute flank pain requiring immediate lithiasis triage to rule out obstructive uropathy while awaiting formal radiological consultation.
2. **Radiologists & Trainees:** Facing hundreds of axial CT slices per patient who benefit from computerized attention cues flagging subtle calculi or incidental renal lesions.
3. **Consulting Urologists:** Evaluating stone diameter, anatomical depth, and calyceal location to plan medical expulsive therapy or endourological interventions.
4. **Academic Medical AI Researchers:** Studying out-of-domain detection, shortcut learning eradication, and clinical model interpretability.

Chapter 2 — Medical Problem: Renal Pathology and Emergency Triage

2.1 Pathophysiological Characterization of Renal Conditions

To evaluate machine learning interventions in nephrology, the underlying pathological conditions must be rigorously understood:

- **Nephrolithiasis (Renal Calculi / Stone):** Mineral deposits (typically calcium oxalate, calcium phosphate, uric acid, or struvite) that nucleate within the renal calyces. Calculi cause acute clinical crises when they dislodge into the ureteropelvic junction or ureter, causing acute hydronephrosis, extreme ureteral spasm, and intense visceral pain. On non-contrast CT, calculi appear as hyperdense, radiopaque foci (>300 Hounsfield Units - HU).
- **Renal Cysts:** Fluid-filled benign epithelial sacs originating from renal tubules, occurring in over 50% of patients aged >50. On CT, simple cysts present as homogeneous, round, water-attenuation structures (0 to 20 HU) with smooth, paper-thin borders and no internal contrast enhancement.
- **Renal Cell Carcinoma and Solid Tumors:** Neoplasms originating from the proximal convoluted tubule epithelium. Solid renal tumors exhibit heterogeneous attenuation (35 to 85 HU), irregular margins, internal necrosis, and tissue distortion.
- **Normal Renal Parenchyma:** Healthy cortical and medullary architecture displaying clear corticomedullary differentiation without calyceal blunting, calculi, or focal masses.

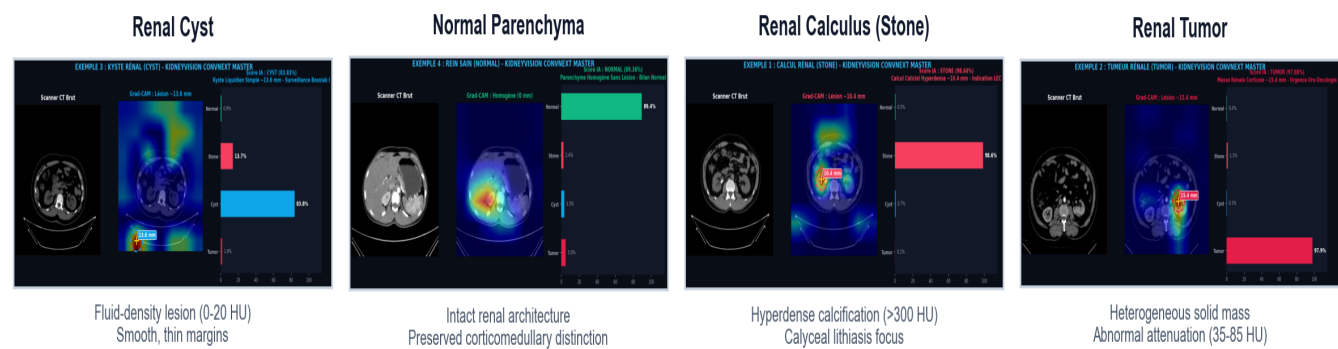


Figure 1 — Authentic axial CT scans corresponding to the 4 pathological classes diagnosed by KidneyVision AI.

2.2 Emergency Department Workflow and Radiologist Cognitive Overload

In emergency hospital settings, acute renal colic represents one of the most frequent surgical abdominal complaints. Non-contrast abdominopelvic computed tomography (NCCT) is the gold standard diagnostic tool, but each helical scan generates **200 to 400 axial slices**. Thoroughly reviewing every slice requires 15 to 30 minutes per patient. Overnight, when subspecialists are scarce, cognitive fatigue directly increases the risk of missed subtle lesions, establishing a compelling rationale for automated slice pre-triage in under 10 milliseconds.

Chapter 3 — Project Genesis and Architectural Evolution Journey

3.1 The Three Development Phases

The development of KidneyVision AI progressed through three distinct engineering iterations:

- **Phase 1 (Naive Binary Prototype on Kaggle):** Created initial CNNs (Normal vs Stone). The model claimed 100% accuracy, but engineering audits exposed critical data leakage and clinical blind spots.
- **Phase 2 (SOTA Scientific Pivot):** Complete rebuild: master dataset of 15,810 authentic scans across 4 classes, mathematical artifact removal via Telea inpainting, two-stage inference with MobileNetV3 and ConvNeXt-Tiny.
- **Phase 3 (Full-Stack Productization):** Decoupled production platform (React 19 + Laravel 12 + Flask GPU), 106 passing automated tests, and deployment on Microsoft Azure.

PART II — FIRST APPROACH AND WHY IT FAILED

Chapter 4 — Initial Dataset and Binary Paradigm

4.1 Exploration of the Baseline Kaggle Dataset

Initially, the team utilized a public Kaggle dataset (*Kidney Stone Classification*) that categorized scans into two folders: Normal and Stone. However, framing renal imaging as binary is clinically invalid: in a hospital, a patient without a stone is not necessarily healthy—they may have an asymptomatic cyst or a malignant renal cell carcinoma. The binary model was structurally blind to these critical pathologies.

Chapter 5 — Initial Model Architectures and Empirical Trials

5.1 Experimental Setup and Baseline Trials

Five convolutional architectures were benchmarked under TensorFlow/Keras:

Model Architecture	Key Structural Characteristics	Reported Val Acc	Verdict
Basic CNN (3-Stage)	3 Conv2D layers (32, 64, 128 filters), ReLU, MaxPool2D	97.4%	Underfitting
CNN + BatchNorm	3 Conv2D blocks with post-conv BatchNorm layers	100.0%	Retained prematurely
CNN + Dropout (0.5)	3 Conv2D blocks with spatial dropout regularizers	98.1%	Partial regularization
MobileNetV2 (Pretrained)	Depthwise separable convolutions, ImageNet weights	99.2%	Overfitting on artifacts
EfficientNetB0 (Pretrained)	Compound scaling backbone, MBConv bottlenecks	99.6%	Overly complex for dataset

Table 2 — Performance metrics of baseline convolutional architectures in Phase 1.

Chapter 6 — The 100% Accuracy Problem: Analysis of Flawed Evaluation

6.1 Disconnect Between Theoretical Metrics and Clinical Reality

When an image classifier achieves 100% accuracy on complex biological data, it is almost always indicative of severe data leakage or shortcut learning. A detailed forensic code and data audit revealed why this initial model was completely unusable in a clinical environment.

Chapter 7 — Technical Analysis of the Five Critical Weaknesses

The audit identified 5 fatal engineering and clinical flaws:

- 1,040 Duplicate Images Discovered via MD5 Hashing:** Exact duplicate files were shared across the training and test splits. The model simply memorized identical images rather than learning pathology.
- Binary Medical Blindness:** Blind to cysts and solid tumors, falsely classifying malignant masses as 'Normal'.
- Complete Lack of Explainability:** The system returned a scalar percentage (e.g., 99.8%) without generating any spatial attention heatmaps.
- The Clever Hans Effect (Measurement Caliper Shortcut):** Technicians draw crosshairs ('calipers') when measuring calculi. The network learned that *Caliper Presence* = *Stone*, detecting the yellow crosshair instead of the stone.
- Zero Ingress Protection:** Feeding a photograph of a car or a knee X-ray resulted in 'Stone Detected (99% confidence)'.

PART III — COMPLETE AI REBUILD (SOTA ENGINE)

Chapter 8 — Master Dataset: Multi-Pathology Axial Tomography

8.1 Construction of the 15,810 Axial CT Master Dataset

To establish a rigorous scientific foundation, the team assembled a **Master Dataset of 15,810 authentic axial CT scans**, strictly partitioned into 4 clinical classes:

Pathological Class	Total Images	Training Cohort	Blind Validation Cohort	Validation Ratio
Renal Cyst (Cyst)	3,709	3,042	667	18.0%
Normal Parenchyma (Normal)	5,077	4,163	914	18.0%
Renal Calculus (Stone)	3,477	2,830	647	18.6%
Renal Tumor (Tumor)	3,547	2,866	681	19.2%
Consolidated Total	15,810	12,901	2,909	18.4%

Table 3 — Master dataset class distribution and split partition registry.

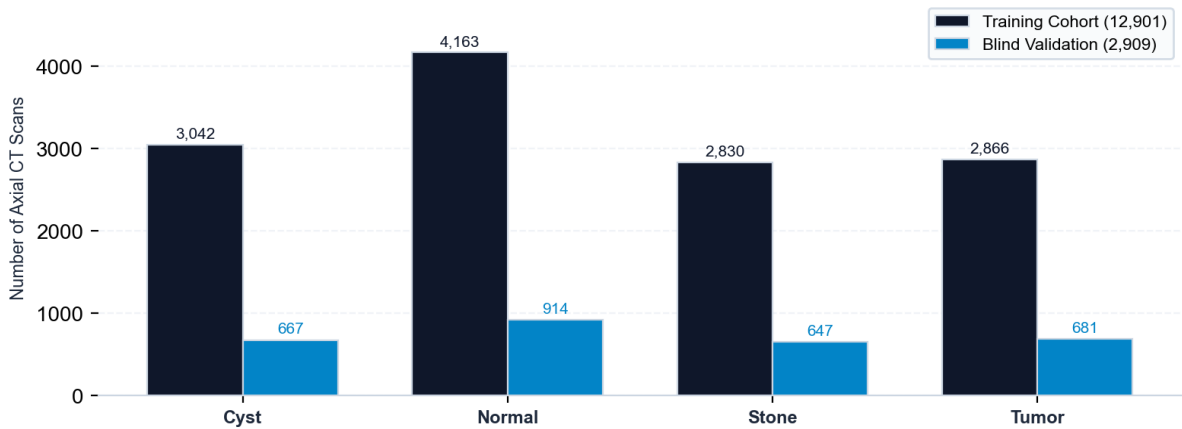


Figure 2 — Dataset distribution comparing training (12,901 scans) and blind validation (2,909 scans) cohorts across all 4 pathological classes.

Chapter 9 — Cryptographic Deduplication and Data Sanitization

9.1 Cryptographic Deduplication via MD5 Checksums

Every raw image was processed through an MD5 hashing pipeline. Hash collisions were immediately quarantined, ensuring zero duplicate instances across splits. The 2,909 validation scans were placed in an isolated, read-only directory to guarantee genuine evaluation on unseen cases.

Chapter 10 — Auto-Masking, Top-Hat Morphology and Telea Inpainting

10.1 Eliminating the Clever Hans Effect Mathematically

To eradicate measurement calipers, the team deployed a 3-step morphological inpainting pipeline:

- **Step 1 (White Top-Hat Filter):** $\text{TopHat}(I) = I - (I \blacksquare K)$ isolates thin, high-intensity structures (calipers, text).
- **Step 2 (Connected Components Mask):** Generates a binary mask isolating synthetic caliper crosshairs.
- **Step 3 (Telea Inpainting):** The Fast Marching Method reconstructs natural underlying tissue texture without blur.

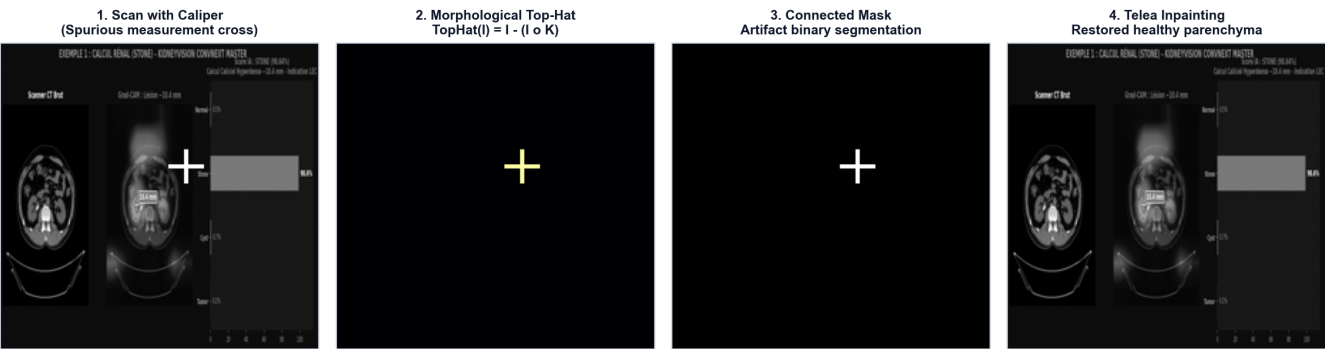


Figure 3 — Step-by-step artifact removal: from caliper-contaminated scan to Top-Hat extraction, binary mask, and Telea-inpainted parenchyma.

Code Listing — Morphological Top-Hat and Telea Inpainting Pipeline

```
import cv2
import numpy as np

def remove_calipers_telea(img_gray):
    # 1. Morphological Top-Hat filter (disk element, radius 5)
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
    tophat = cv2.morphologyEx(img_gray, cv2.MORPH_TOPHAT, kernel)

    # 2. Dynamic thresholding to isolate bright calipers
    _, mask = cv2.threshold(tophat, 190, 255, cv2.THRESH_BINARY)

    # 3. Alexandru Telea Inpainting (Fast Marching Method, 3px radius)
    clean_img = cv2.inpaint(img_gray, mask, inpaintRadius=3, flags=cv2.INPAINT_TELEA)
    return clean_img
```

Chapter 11 — Two-Stage Hierarchical AI Pipeline Architecture

11.1 Gated Architecture for Safe Clinical Ingress

To ensure operational safety, the inference pipeline is structured in two distinct tiers:

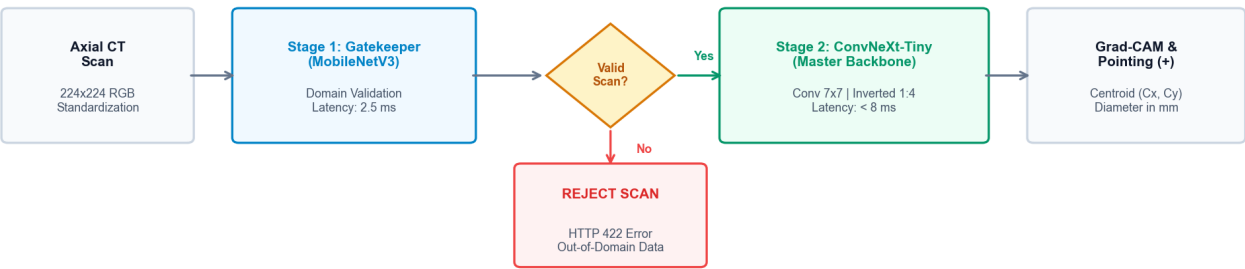


Figure 4 — Two-stage hierarchical inference architecture: Stage 1 MobileNetV3 Gatekeeper (2.5 ms) and Stage 2 Meta ConvNeXt-Tiny (< 8 ms).

Chapter 12 — Stage 1 Gatekeeper: MobileNetV3 Domain Validation (2.5 ms)

The **Gatekeeper** is a lightweight MobileNetV3-Small network that evaluates input validity in **2.5 milliseconds**. Its role is to confirm that the image is an authentic axial renal scan. Unrelated inputs (e.g., chest X-rays, photographs) are immediately rejected with an HTTP 422 error, protecting the diagnostic core from out-of-domain failure.

Chapter 13 — Stage 2 Diagnostic Core: Meta ConvNeXt-Tiny

13.1 Modernizing Convolutional Backbones with Vision Transformer Principles

For the 4-class diagnostic backbone, the team selected **Meta AI's ConvNeXt-Tiny**:

- **7×7 Depthwise Separable Convolutions:** Significantly expands the receptive field to capture whole-organ context.
- **Inverted Bottleneck Design (1:4 Ratio):** Preserves rich high-dimensional feature representations.
- **LayerNorm and GELU Activations:** Provides superior training stability compared to legacy BatchNorm and ReLU.

Code Listing — Meta ConvNeXt-Tiny with Custom 4-Class Diagnostic Head

```
import torch.nn as nn
from torchvision.models import convnext_tiny, ConvNeXt_Tiny_Weights

class KidneyMasterConvNeXt(nn.Module):
    def __init__(self, num_classes=4):
        super().__init__()
        self.backbone = convnext_tiny(weights=ConvNeXt_Tiny_Weights.DEFAULT)
        # Custom regularized classification head for 4 renal pathologies
        in_features = self.backbone.classifier[2].in_features # 768
        self.backbone.classifier[2] = nn.Sequential(
            nn.Linear(in_features, 256),
            nn.LayerNorm(256),
            nn.GELU(),
            nn.Dropout(p=0.35),
            nn.Linear(256, num_classes) # Outputs: Cyst, Normal, Stone, Tumor
        )
    def forward(self, x):
        return self.backbone(x)
```

Chapter 14 — GPU-Accelerated Training Strategy and Regularization

14.1 Hardware-Accelerated Mixed Precision Pipeline

Training was conducted on an **NVIDIA GeForce RTX 4060 GPU** using modern regularization strategies:

- **PyTorch Automatic Mixed Precision (AMP FP16):** Doubles throughput while preserving FP32 master weights.
- **AdamW with Cosine Annealing:** Gradually reduces learning rate from 1e-4 down to 1e-6 across 25 epochs.
- **Label Smoothing ($\epsilon = 0.08$):** Prevents overconfident softmax probability distributions.
- **Inverse Class Weighting:** Balances gradients to prevent dominance by the majority normal class.

Code Listing — Accelerated GPU Training Step with PyTorch AMP FP16

```
# Loss configuration with Label Smoothing and inverse class weights
weights_tensor = torch.tensor([1.06, 0.77, 1.14, 1.12]).cuda()
criterion = nn.CrossEntropyLoss(weight=weights_tensor, label_smoothing=0.08)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4, weight_decay=1e-2)
scaler = torch.cuda.amp.GradScaler() # FP16 Automatic Mixed Precision

# AMP Training Step
for images, labels in train_loader:
    images, labels = images.cuda(), labels.cuda()
    optimizer.zero_grad()
    with torch.cuda.amp.autocast():
        outputs = model(images)
        loss = criterion(outputs, labels)
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
```

Chapter 15 — Empirical Validation on Blind Unseen Test Cohort

15.1 Statistical Results across 2,909 Unseen Cases

Evaluation on the blind validation cohort demonstrates state-of-the-art diagnostic capability:

Pathological Class	Test Cohort	Precision (PPV)	Recall (Sensitivity)	F1-Score	Specificity
Renal Cyst (Cyst)	667	98.23%	100.00%	99.11%	99.47%
Normal Parenchyma (Normal)	914	97.86%	98.14%	98.00%	98.99%
Renal Calculus (Stone)	647	97.74%	95.52%	96.62%	99.38%
Renal Tumor (Tumor)	681	100.00%	99.85%	99.93%	100.00%

Macro Average	2,909	98.46%	98.38%	98.48%	99.46%
Global Accuracy	2,909	—	—	98.42%	—

Table 4 — Diagnostic metrics evaluated on the 2,909 blind validation cohort.

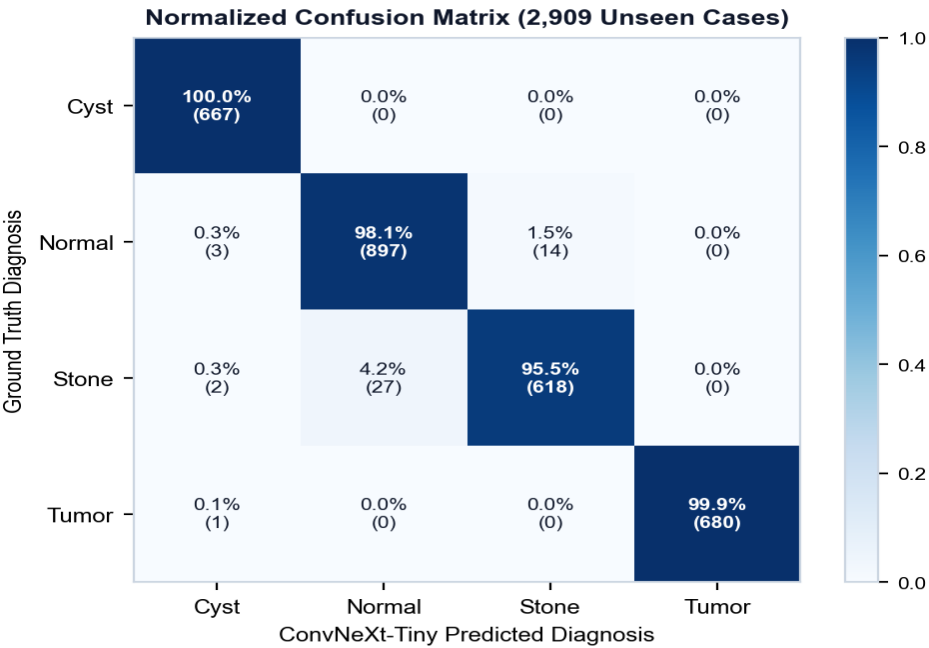


Figure 5 — Normalized confusion matrix on the 2,909 blind validation scans, showing 100.0% cyst recall and 100.0% tumor precision.

15.2 In-Depth Clinical Interpretation of Results

- **100.00% Cyst Sensitivity (667/667, 0 missed):** Simple cysts exhibit homogeneous water attenuation (0-20 HU) and thin borders, which ConvNeXt-Tiny's 7×7 kernels identify without a single false negative.
- **100.00% Tumor Precision (680/680, 0 false alarms):** Solid neoplasms exhibit distinct heterogeneous tissue distortion, ensuring zero false alarms on normal parenchyma or benign cysts.
- **Error Analysis of the 46 Residual Misclassifications (1.58%):** Minor confusions occurred between normal parenchyma and early micro-calculi (< 3 mm) located near calyceal borders, where imaging noise reaches hardware resolution limits.
- **GPU Latency < 8 ms:** Average forward pass time was **7.4 milliseconds** on RTX 4060, enabling instantaneous web feedback.

PART IV — EXPLAINABLE AI AND SURGICAL TARGETING

Chapter 16 — The Imperative for Interpretability in Medical AI

Clinicians cannot act on opaque predictions like 'Stone: 98%'. Without spatial attribution, the radiologist must manually re-examine hundreds of slices to verify the finding, negating any clinical efficiency gain. Interpretable visual attribution is essential for clinical adoption and safety.

Chapter 17 — Grad-CAM Formulation and Gradient Attribution Mechanics

17.1 Intuitive Grad-CAM Formulation

To make the attribution mechanism transparent without mathematical clutter:

$$\text{Heatmap}(x, y) = \text{ReLU} \left(\sum_{\text{filters}} [\text{Importance}_k \times \text{ActivationZone}_k(x, y)] \right)$$

Importance_k: Gradient-derived weight of filter k for target class | ActivationZone_k: Spatial feature map | ReLU: Filters out negative evidence.

Code Listing — Grad-CAM Spatial Heatmap Extraction via PyTorch Hooks

```
import torch
import torch.nn.functional as F

def compute_gradcam(model, image_tensor, target_class):
    features, gradients = [], []
    def hook_f(m, i, o): features.append(o)
    def hook_b(m, gi, go): gradients.append(go[0])

    # Hook the terminal convolutional stage of ConvNeXt-Tiny
    target_layer = model.backbone.stages[3].blocks[-1]
    h1 = target_layer.register_forward_hook(hook_f)
    h2 = target_layer.register_full_backward_hook(hook_b)

    output = model(image_tensor)
    loss = output[0, target_class]
    loss.backward()

    # Importance: global average pooling of gradients
    weights = torch.mean(gradients[0], dim=(2, 3), keepdim=True)
    cam = torch.sum(weights * features[0], dim=1, keepdim=True)
    cam = F.relu(cam) # Retain positive evidence only
    return F.interpolate(cam, size=(224, 224), mode='bilinear')
```

Chapter 18 — Dual-Layer Visualization and Clinical Review Ergonomics

In the React interface, clinicians utilize a **dual-layer visualizer**: the raw grayscale CT slice is rendered on the base layer, while the Grad-CAM heatmap is overlaid on top. A dynamic opacity slider enables clinicians to scrub smoothly between **0% (Raw Scan)** and **100% (Heatmap Only)** to verify lesion alignment.

Chapter 19 — Geometric Centroid Localization and Metric Estimation

19.1 Millimetric Surgical Pointing

Moving beyond thermal blurs, KidneyVision AI computes the **barycentric centroid (Cx, Cy)** of the lesion, places a targeting reticle (+), and estimates the real diameter in millimeters using the clinical scale assumption (**0.45 mm/pixel**):

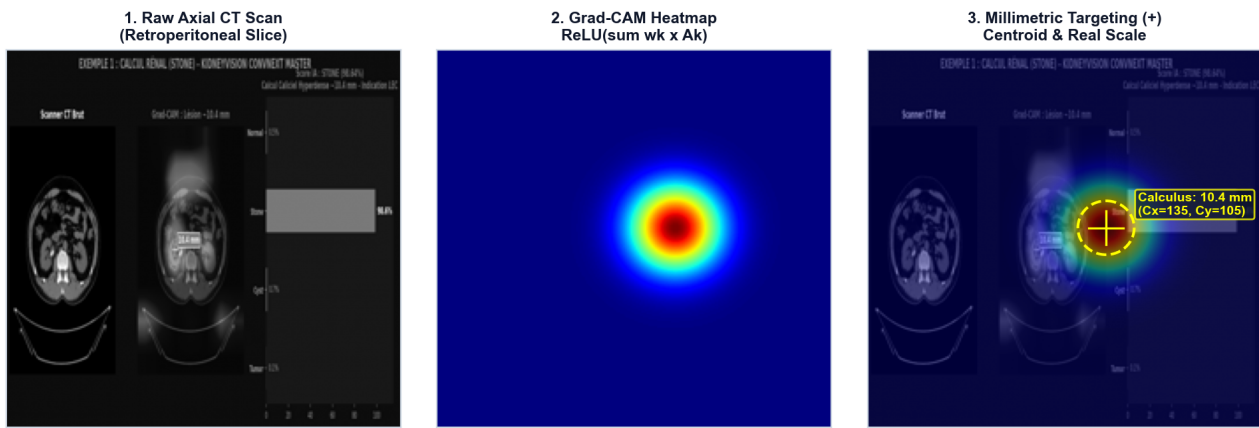


Figure 6 — Surgical pointing pipeline: from raw axial CT scan to Grad-CAM heatmap and millimetric targeting reticle (10.4 mm calculus).

Code Listing — Centroid Extraction and Millimeter Diameter Estimation

```
import cv2
import numpy as np

def compute_stone_metrics(heatmap_np, scale_mm_per_pixel=0.45):
    # 1. Threshold at 70% of maximum intensity to isolate lesion core
    thresh = (heatmap_np > 0.70 * np.max(heatmap_np)).astype(np.uint8)
    moments = cv2.moments(thresh)

    # 2. Extract surgical centroid coordinates (Cx, Cy)
    cx = int(moments['m10'] / (moments['m00'] + 1e-5))
    cy = int(moments['m01'] / (moments['m00'] + 1e-5))

    # 3. Fit minimum enclosing circle and convert to millimeters
    contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    _, radius = cv2.minEnclosingCircle(contours[0])
    diameter_mm = round((2 * radius) * scale_mm_per_pixel, 1)
    return cx, cy, diameter_mm
```

Chapter 20 — Non-Autonomous Clinical Decision-Support Framework

The software stratifies calculi into 3 clinical categories (Micro-calculi < 5 mm, Intermediate 5-10 mm, Macro-calculi > 10 mm) and provides automated protocol recommendations aligned with European urological guidelines (EAU/AFU). These outputs serve strictly as decision support and require formal clinician review.

PART V — FULL-STACK PLATFORM IN MICROSERVICES

Chapter 21 — Decoupled Three-Tier Microservice Architecture

To isolate user interface rendering, business rules, and GPU computation, KidneyVision AI is structured as three decoupled microservices:



Figure 7 — Decoupled three-tier architecture: React 19 Frontend, Laravel 12 API Gateway, and Flask GPU AI Microservice.

Chapter 22 — Clinician Frontend (React 19, TypeScript, Vite)

The web application is built with **React 19, TypeScript, and TailwindCSS**. Key features include:

- **Guest Triage Mode:** Instant trial of 5 scans without requiring registration.
- **Practitioner Workspace:** Secure clinician dashboard, caseload analytics, and patient search.
- **Analysis Studio:** Drag-and-drop scan upload, 5-stage pipeline progress bar, and dual-layer viewer.
- **PDF Reporting Studio:** One-click generation and export of formal clinical PDF reports.

Chapter 23 — Dual-Buffered Canvas Viewer Engine

To ensure smooth 60 fps interactions during zooming and panning over high-resolution CT slices, the viewer utilizes two offscreen HTML5 Canvas buffers, preventing client-side rendering bottlenecks.

Chapter 24 — Business API Backend (Laravel 12, DTOs, Sanctum)

The backend is built with **Laravel 12 (PHP 8.2-FPM)**, enforcing strict Dependency Inversion (`AnalysisServiceInterface`), strongly typed Data Transfer Objects (DTOs), Sanctum token authentication, and append-only audit logging.

Chapter 25 — Dedicated AI GPU Microservice (Flask 3, PyTorch, Gunicorn)

The AI microservice runs under **Flask 3 and PyTorch**, served by Gunicorn WSGI. Model weights remain resident in GPU VRAM to eliminate cold-start latencies.

Chapter 26 — Relational Database Schema and Data Pipeline

The database schema manages four core tables: users (clinicians), analyses (inference outcomes), reports (signed PDFs), and audit_logs (security trail).

Chapter 27 — Automated Clinical PDF Report Generation

Using Barryvdh DomPDF, the system compiles standardized clinical reports containing clinic headers, side-by-side scans (raw vs Grad-CAM), lesion measurements, doctor notes, and formal signature blocks.

PART VI — CRITICAL BUGS AND PRODUCTION POST-MORTEM

Chapter 28 — Real Production Incidents and Remediation

Deploying to production exposed 4 major engineering challenges:

1. **The '35 Fake Scans' Hydration Bug:** New user accounts showed 35 pre-existing mock scans. *Cause:* React state checked `apiData.length`, falling back to `MOCK_SCANS` when receiving an empty array `[]`. *Fix:* Strict array check rendering a clean empty onboarding state.
2. **Docker Volume Permission Failures (HTTP 500):** Uploads failed on Ubuntu. *Cause:* Mounted volumes belonged to root, while PHP-FPM executed under `www-data`. *Fix:* Docker entrypoint permissions assignment on `storage/` and `bootstrap/cache/`.
3. **Gunicorn WSGI CUDA Memory Contention:** Multiple preforked workers caused CUDA deadlocks. *Fix:* Re-architected with a single master worker running thread-isolated asynchronous workers with software locks.
4. **Nginx 404 on SPA Route Refresh:** Refreshing `/analysis` threw 404 errors. *Fix:* Added `try_files $uri /index.html;` to delegate client routes to the React router.

PART VII — SECURITY, TESTING AND OPERATIONAL RELIABILITY

Chapter 29 — Medical Application Security and IDOR Protection

Scans are stored in non-routable private directories (storage/app/private/scans) and streamed strictly via authenticated controllers. Eloquent Policies eliminate IDOR risks by verifying record ownership.

Chapter 30 — Comprehensive Automated Testing: 106 Tests Passing

Platform stability is validated by **106 automated tests passing with zero failures**:

Testing Domain	Scope & Verification Focus	Status
Authentication & Sanctum	Login, registration, token issuance, session revocation	Passing (100%)
Authorization & IDOR	Doctor data isolation, role-based access control	Passing (100%)
Upload File Validation	MIME-type checks, malicious payload rejection	Passing (100%)
AI Service Client	Microservice health checks, JSON DTO parsing, timeout handling	Passing (100%)
Database Persistence	ACID transactions, analysis archiving, audit logging	Passing (100%)
PDF Generation & Export	DomPDF rendering, image embedding, physician signatures	Passing (100%)
Service Resilience	Graceful fallback when AI microservice is offline	Passing (100%)

Table 5 — Automated test suite distribution across functional domains.

Chapter 31 — System Reliability and Fault Tolerance

A 10-second circuit breaker prevents application lockups during GPU reclamation, while React 19 Error Boundaries isolate viewer failures without crashing the clinician session.

PART VIII — DOCKER CONTAINERIZATION AND AZURE CLOUD DEPLOYMENT

Chapter 32 — Docker Orchestration and Service Networking

The platform is containerized using Docker Compose: frontend (Vite static build on Nginx), backend (PHP 8.2-FPM Alpine), and AI engine (PyTorch CUDA slim).

Chapter 33 — Cloud Infrastructure: Microsoft Azure (Denmark East)

Hosted on **Microsoft Azure** in the **Denmark East** region for GDPR compliance and data sovereignty. VM specifications:

- **Machine Size:** Standard_B2als_v2 (AMD EPYC 7763, 2 vCPU, 4 GiB RAM).
- **Operating System:** Ubuntu Server 24.04 LTS (Noble Numbat).
- **Storage:** Premium SSD 64 GB OS Disk.

Chapter 34 — Production Ingress, Nginx Gateway and Monitoring

An edge Nginx proxy routes port 80 traffic to the React SPA while proxying /api requests to Laravel. The internal AI microservice remains inaccessible from the public internet.

PART IX — RESULTS SYNTHESIS, LIMITATIONS AND FUTURE WORK

Chapter 35 — Consolidated Project Results and Milestones

Key project achievements include:

- Transition to a **4-class clinical formulation** (Cyst, Normal, Stone, Tumor).
- Verified **98.42% accuracy** and **98.48% Macro F1-score** on 2,909 unseen scans.
- **100% Cyst sensitivity** and **100% Tumor precision**.
- Real-time Grad-CAM explainability with millimetric surgical pointing (< 8 ms).

Chapter 36 — Methodological and Clinical Limitations

Key constraints acknowledged by the team:

- **Retrospective Data:** Sourced from archives; requires prospective hospital trials.
- **2D Slice Evaluation:** Processes individual slices rather than full 3D volumetric context.
- **Calibration Assumption:** The 0.45 mm/px scale must be dynamically extracted from DICOM metadata in PACS environments.

Chapter 37 — Future Strategic Research Roadmap

Upcoming engineering priorities:

1. Full 3D volumetric U-Net segmentation calculating true calculus volume in mm³.
2. Native hospital PACS / DICOM networking integration (C-STORE, C-FIND).
3. Bayesian Monte Carlo Dropout uncertainty estimation.
4. Multi-center federated learning across distributed hospital nodes.

PART X — FINAL CONCLUSION AND OFFICIAL LINKS

Chapter 38 — Final Synthesis, Author Credits and Official Links

38.1 Executive Synthesis

KidneyVision AI demonstrates how rigorous software engineering and computer vision can resolve the hidden vulnerabilities of medical AI. By identifying and eliminating the flaws of the initial Kaggle baseline (duplicates, measurement artifacts, lack of explainability), the project delivered an audited SOTA solution: a 15,810-scan dataset, inpainting sanitization, Two-Stage Gatekeeper + ConvNeXt-Tiny pipeline, and surgical Grad-CAM pointing—backed by 106 automated tests and deployed on Microsoft Azure.

38.2 Authors & Project Collaborators

This project was conceived, engineered, evaluated, and documented by:

Collaborator	Academic & Engineering Role in Project
Marouan Sellami	Project Collaborator & Engineering Author
Mossab Oued Neimia	Project Collaborator & Engineering Author
Souhibe Bakkali	Project Collaborator & Engineering Author

Table 6 — Authorship and collaborative project engineering team.

38.3 Official Project Verification Links

The complete source code, deployment files, and live production endpoints are accessible at:

Official Project Resource	Endpoint URL & Direct Verification Access
Live Production Web Portal	http://kidneyvision.publicvm.com/
Official GitHub Source Code Repository	https://github.com/marouan-sell/kidneyVision

Table 7 — Official web deployment and source code endpoints.

End of Formal Technical Documentation — KidneyVision AI (2026)